

CLAUDE CODE & OPENCODE

SICHER UND PRODUKTIV EINSETZEN

2 WOCHEN VORHER

Vorbereitungsgespräch — welche Use Cases und Codebeispiele aus Ihrem Tagesgeschäft in den Workshop einfließen

1 WOCHE VORHER

Remote-Setup — Installation gemeinsam mit allen Teilnehmern, jeder am Veranstaltungstag startklar

VERANSTALTUNGSTAG

Der Halbtagesworkshop — Module A–C am eigenen Code (Beamer oder Display mit USB-C / HDMI)

1 WOCHE DANACH

90-Min Praxis-Recap — die interessantesten Themen aus der Praxiswoche live und gemeinsam gelöst

Was nehmen Sie aus dem Workshop mit?

- Effizientes Vibe Coding (damit wir bekommen, was Sie wollen)
- Ein geregelter Ablauf (etwas Neues bekommen, ohne das Alte zu verlieren)
- Autonomes Programmieren (ohne dass dabei etwas schief läuft)

- MODUL A **SICHER LOSLEGEN (ERSTE SCHRITTE)**
- MODUL B **PROJEKTGEDÄCHTNIS (KONFIGURATION)**
- MODUL C **SKILLS & AUTOMATISCHE LEITPLANKEN**
- HANDS-ON-PROGRAMMING MIT SPN BEISPIELEN**

MODUL A

SICHER LOSLEGEN

#1

CLAUDE CODE **VS.** OPENCODE



CLAUDE CODE

Perfekt für den Einstieg

- Von Anthropic, optimiert auf Claude-Modelle
- Einfacher Einstieg, native Integration in IDEs
- Reifes Ökosystem mit Skills, Hooks, Subagents

OPENCODE

Ideal zur Individualisierung

- Open Source
- Modell-agnostisch
- Individualisierbar

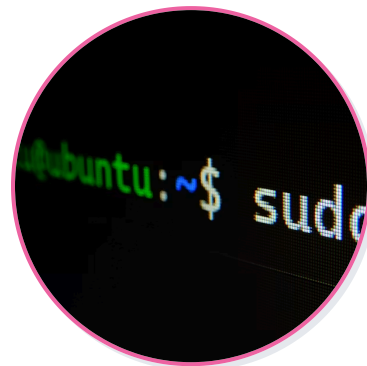
Claude Code so konfigurieren, dass keine Daten unkontrolliert abfließen.

- Anthropic API ersetzen, z.B. Modelle im Green Data Center der LEW
- Telemetrie und Daten-Sharing deaktivieren
- Alternativ sensible Dateien und Secrets aus dem Kontext halten

```
# Claude Code auf ein lokales Modell umleiten
$ export ANTHROPIC_BASE_URL="http://api.openhippo.io"
$ export ANTHROPIC_AUTH_TOKEN="$HIPPO_TOKEN"
$ export ANTHROPIC_MODEL="hippo-chat-large"
$ export ANTHROPIC_SMALL_FAST_MODEL="hippo-chat-large"
$ export DISABLE_TELEMETRY=1
$ claude
```

Vom leeren Terminal zum ersten echten Code.

- Claude Code oder OpenCode, was passt zu mir?
- Wie konfiguriere ich meinen Helfer (z.B. KI-Modelle auswählen)?
- Was sieht die KI und wie informiert sie sich?



Was das Tool weiß, entscheidet über das Ergebnis.

- Was landet im Kontext?
- Effizient bleiben: Ein Chat für eine Aufgabe.
- **Headroom** — das Kontextfenster bewusst nicht überfüllen
- **Codegraph** — bei großen Projekten nur relevante Dateien über den Abhängigkeitsgraphen laden



REMINDER Je spezialisierter der Kontext, umso besser das Ergebnis.

Wie BAUE ICH mir eine APP

- 1 Design
- 2 Claude Code / OpenCode Setup
- 3 In kleinen Schritten bauen
- 4 Testen, prüfen, ausliefern



REMINDER Human-in-the-Loop

MODUL B

PROJEKTGEDÄCHTNIS

#2

WAS IM GEDÄCHTNIS BLEIBEN SOLLTE



An einem Ort — beim nächsten Aufruf sofort verfügbar.

- Projektstruktur
- Architekturentscheidungen
- Sicherheitsregeln
- Coding Standards (z.B. git)
- Richtlinien für CI/CD



TIPP Die KI wie einen Junior Developer bei einem Projekt onboarden.

WO LIEGT DAS GEDÄCHTNIS?



Eine Datei — beide Werkzeuge lesen sie.

CLAUDE CODE

- CLAUDE.md im Projektwurzelverzeichnis
- Wird bei jedem Start automatisch geladen
- Kann von der KI aktualisiert werden

OPENCODE

- Nutzt standardmäßig AGENTS.md
- Liest CLAUDE.md am selben Ort
- Funktioniert gleich

MERKE Im Zweifel bei mehreren Entwicklern CLAUDE.md als Standard setzen.

WIE MACHE ICH ein PROJEKTGEDÄCHTNIS?



KI nutzen, um KI zu konfigurieren — nicht von Hand schreiben.

- Von der KI generieren lassen (/init) statt auf der leeren Seite anfangen
- Im Alltag verfeinern: „Merk dir das in CLAUDE.md" nach guten und schlechten Antworten
- Die KI ihre eigenen Regeln zusammenfassen und kürzen lassen
- Kurz & aktuell halten — mit dem Projekt mitwachsen lassen

MERKE Lass die KI ihr eigenes Gedächtnis schreiben — du redigierst nur.

MODUL C

SKILLS & LEITPLANKEN

#3

AGENTS

Delegieren

- Halten den High-Level-Prompt sauber
- Orchestrieren Teilaufgaben von A bis Z

COMMANDS

Wiederholen

- Wiederkehrende Aufgaben als fester Befehl
- Konsistent — kein Varianz-Risiko im Ergebnis

SKILLS

Können

- Bündeln Fachwissen als wiederverwendbare Fähigkeit
- Werden bei Bedarf automatisch geladen

HOOKS

Erzwingen

- Prüfung bei jedem Werkzeugaufruf
- Regeln deterministisch durchgesetzt

MERKE Lass die Bausteine von der KI erstellen — das kann sie besser.

Eigene Erweiterungen — abgelegt in der Ordnerstruktur.

.CLAUDE/

Claude Code

- commands/ — eigene Befehle
- agents/ — spezialisierte Subagents
- hooks & settings.json — Leitplanken

.AGENTS/

OpenCode

- AGENTS.md als Einstiegspunkt
- agent/ — eigene Agents
- command/ — eigene Commands

Ein kleiner Kreislauf, den man immer wieder durchläuft.



...und wieder von vorn.

FRAGE Wie organisiere ich die Arbeit?

In einer isolierten Umgebung darf die KI ohne Rückfragen arbeiten.

- Docker-Container: getrennt vom Host, kein Zugriff aufs echte System
- Nur das Projekt gemountet — Schaden bleibt eingesperrt und ist wegwerfbar

```
# NUR im isolierten Container ausführen
$ docker run -it --rm -v $PWD:/app sandbox

# Claude ohne Rückfragen
$ claude --dangerously-skip-permissions

# OpenCode: Rechte in opencode.json auf "allow"
$ opencode
```

WAS *JeTzT*? — MACHen!